Week 4

SQL Queries
(Continued)

Recap

- **Data Definition Language**
- DDL use:
  - Create (create database, create table)
  - Drop
  - Alter

CREATE DATABASE *databasename*;

Recap

The CREATE TABLE Command

- **Syntax:**

```
CREATE TABLE table_name (
 column1  datatype,
 column2 datatype,
 column3 datatype,
....
 );
```

- **Example:**

```
CREATE TABLE Persons (
PersonID int,
LastName varchar(255),
FirstName varchar(255),
Address varchar(255),
);
```

Recap

- show databases; shows all databases.
- use databasename; database that u want to work with.
- show tables; shows all tables in the database
- describe tablename; shows structure of a table

Data Manipulation Language

- A DML statements are used to:
  - Add new rows to a table – Insert statement
  - Modify existing rows in a table – Update statement
  - Remove existing rows from a table – Delete statement

Insert a New Row to a Table

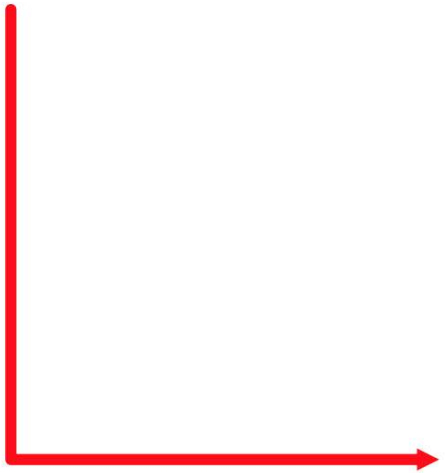
70	Public Relations	100	1700

New row

DEPARTMENTS

DEPARTMENT_ID	DEPARTMENT_NAME	MANAGER_ID	LOCATION_ID
10	Administration	200	1700
20	Marketing	201	1800
50	Shipping	124	1500
60	IT	103	1400
80	Sales	149	2500
90	Executive	100	1700
110	Accounting	205	1700
190	Contracting		1700

Insert new row into the DEPARTMENTS table



DEPARTMENT_ID	DEPARTMENT_NAME	MANAGER_ID	LOCATION_ID
10	Administration	200	1700
20	Marketing	201	1800
50	Shipping	124	1500
60	IT	103	1400
80	Sales	149	2500
90	Executive	100	1700
110	Accounting	205	1700
190	Contracting		1700
70	Public Relations	100	1700

INSERT INTO Statement

- To add a **new row** to a table by using the **INSERT INTO** statement:

```
INSERT INTO table_Name
VALUES      ('value1','value2', 'value3',…… );
```

- With this syntax, only one row is inserted at a time.

Example

- Employees table (emp_id, first_name, last_name, dept_id)
- Insert into Employees values (1, ‘john’, ‘smith’, 4);
- To insert multiple rows together:
- Insert into Employees values (2, ‘alice’, ‘brown’, 4), (3, ‘joyce’, ‘popp’, 2), (4, ‘louis’, ‘smith’, 3);
- Char or date values are enclosed in single quotation marks ‘’
- You should list values according to order of columns in table.

To insert in bulk from csv files

- LOAD DATA INFILE ’filename.csv’ INTO TABLE Employees FIELDS TERMINATED BY ',’ LINES TERMINATED BY '\n’ IGNORE 1 ROWS (emp_id, first_name, last_name, dept_id);
- **INFILE = input file**.
- It tells MySQL: *“Read data from this external file (CSV or text) and load it into a table.”*

- **FIELDS TERMINATED BY ','**
- defines how columns are **separated** inside the file.
- In CSV (Comma-Separated Values):
- FIELDS TERMINATED BY ','
- If the file used tabs:
- FIELDS TERMINATED BY '\t'

Import files in workbench(GUI)

- **Open MySQL Workbench**
- Connect to your MySQL server.
- **Select Your Database (Schema)**
- In the **Navigator (left panel)**, expand your schema.
- Right-click on the **table** (e.g., Employees).
- Choose **Table Data Import Wizard**.
- **Choose the File**
- Browse to your employees.csv.

Inserting Special Values

- The **SYSDATE** function records the current date and time.

```
INSERT INTO employees
VALUES (113, 'Louis', 'Popp', 'LPOPP', '515.124.4567',
                  SYSDATE, 'AC_ACCOUNT', 6900,
                  NULL, 205, 100);
1 row created.
```

Inserting Specific Date Values

- Add a new employee.

```
INSERT INTO employees
VALUES      (114,
             'Den', 'Raphealy',
             'DRAPHEAL', '515.127.4561',
             TO_DATE('FEB 3, 1999', 'MON DD, YYYY'),
             'AC_ACCOUNT', 11000, NULL, 100, 30);
1 row created.
```

to convert the string
'FEB 3, 1999' to a date value

- Verify your addition

EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_P
114	Den	Raphealy	DRAPHEAL	515.127.4561	03-FEB-99	AC_ACCOUNT	11000	

Example

- Table Customers

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
89	White Clover Markets	Karl Jablonski	305 - 14th Ave. S. Suite 3B	Seattle	98128	USA
90	Wilman Kala	Matti Karttunen	Keskuskatu 45	Helsinki	21240	Finland
91	Wolski	Zbyszek	ul. Filtrowa 68	Walla	01-012	Poland

- Add new customer

Example

- **INSERT INTO** Customers
  VALUES (92, 'Cardinal', 'Tom B. Erichsen', 'Skagen 21', 'Stavanger’,

'4006', 'Norway');

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
89	White Clover Markets	Karl Jablonski	305 - 14th Ave. S. Suite 3B	Seattle	98128	USA
90	Wilman Kala	Matti Karttunen	Keskuskatu 45	Helsinki	21240	Finland
91	Wolski	Zbyszek	ul. Filtrowa 68	Walla	01-012	Poland
92	Cardinal	Tom B. Erichsen	Skagen 21	Stavanger	4006	Norway

Update Data in a Table

EMPLOYEES

EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	HIRE_DATE	JOB_ID	SALARY	DEPARTMENT_ID	COMMISSION_P
100	Steven	King	SKING	17-JUN-87	AD_PRES	24000	90	
101	Neena	Kochhar	NKOCHHAR	21-SEP-89	AD_VP	17000	90	
102	Lex	De Haan	LDEHAAN	13-JAN-93	AD_VP	17000	90	
103	Alexander	Hunold	AHUNOLD	03-JAN-90	IT_PROG	9000	60	
104	Bruce	Ernst	BERNST	21-MAY-91	IT_PROG	6000	60	
107	Diana	Lorentz	DLORENTZ	07-FEB-99	IT_PROG	4200	60	
124	Kevin	Mourgos	KMOURGOS	16-NOV-99	ST_MAN	5800	50	

Update rows in the **EMPLOYEES** table:

EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	HIRE_DATE	JOB_ID	SALARY	DEPARTMENT_ID	COMMISSIO
100	Steven	King	SKING	17-JUN-87	AD_PRES	24000	90	
101	Neena	Kochhar	NKOCHHAR	21-SEP-89	AD_VP	17000	90	
102	Lex	De Haan	LDEHAAN	13-JAN-93	AD_VP	17000	90	
103	Alexander	Hunold	AHUNOLD	03-JAN-90	IT_PROG	9000	30	
104	Bruce	Ernst	BERNST	21-MAY-91	IT_PROG	6000	30	
107	Diana	Lorentz	DLORENTZ	07-FEB-99	IT_PROG	4200	30	
124	Kevin	Mourgos	KMOURGOS	16-NOV-99	ST_MAN	5800	50	

UPDATE Statement

- Modify existing rows with the **UPDATE** statement:

```
UPDATE      table_Name
SET         column1 = value1, column2 = value2, ...
WHERE       condition;
```

- Update more than one row at a time (if required).

Update Statement

- Specific row is modified if you specify the `WHERE` clause:

```
UPDATE employees
SET    department_id = 70
WHERE  employee_id = 113;
1 row updated.
```

- All rows in the table are modified if you omit the `WHERE` clause:

```
UPDATE   copy_emp
SET      department_id = 110;
22 rows updated.
```

- Updates the first customer (`CustomerID=1`) with a new contact person and a new city

```
UPDATE Customers
SET ContactName = 'Alfred Schmidt', City= 'Frankfurt'
WHERE CustomerID = 1;
```

Example

- Table Customers

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin	12209	Germany
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.	05023	Mexico
4	Around the Horn	Thomas Hardy	120 Hanover Sq.	London	WA1 1DP	UK
5	Berglunds snabbköp	Christina Berglund	Berguvsvägen 8	Luleå	S-958 22	Sweden

- Update first customer (CustomerID = 1) with a new contactname *and* a new city.

Example

- **UPDATE** Customers
  SET ContactName = 'Alfred Schmidt',
  City= 'Frankfurt'
  WHERE CustomerID = 1;

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Alfred Schmidt	Obere Str. 57	Frankfurt	12209	Germany
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.	05023	Mexico
4	Around the Horn	Thomas Hardy	120 Hanover Sq.	London	WA1 1DP	UK
5	Berglunds snabbköp	Christina Berglund	Berguvsvägen 8	Luleå	S-958 22	Sweden

Example

- **UPDATE** Customers
  SET ContactName='Juan'
  WHERE Country='Mexico'

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Alfred Schmidt	Obere Str. 57	Frankfurt	12209	Germany
2	Ana Trujillo Emparedados y helados	Juan	Avda. de la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taquería	Juan	Mataderos 2312	México D.F.	05023	Mexico
4	Around the Horn	Thomas Hardy	120 Hanover Sq.	London	WA1 1DP	UK
5	Berglunds snabbköp	Christina Berglund	Berguvsvägen 8	Luleå	S-958 22	Sweden

Example

- **UPDATE** Customers
  SET ContactName='Juan';

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Juan	Obere Str. 57	Frankfurt	12209	Germany
2	Ana Trujillo Emparedados y helados	Juan	Avda. de la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taquería	Juan	Mataderos 2312	México D.F.	05023	Mexico
4	Around the Horn	Juan	120 Hanover Sq.	London	WA1 1DP	UK
5	Berglunds snabbköp	Juan	Berguvsvägen 8	Luleå	S-958 22	Sweden

DELETE Statement

- Syntax

```
DELETE FROM table_name WHERE condition;
```

- **Note:** Be careful when deleting records in a table! Notice the WHERE clause in the DELETE statement.
- The WHERE clause specifies which record(s) should be deleted.
- If you omit the WHERE clause, all records in the table will be deleted!

DELETE Examples

- To deletes the customer "Alfreds Futterkiste" from the "Customers" table

```
DELETE FROM Customers
WHERE CustomerName='Alfreds Futterkiste';
```

- To deletes all rows in the "Customers" table, without deleting the table

```
DELETE FROM Customers;
```

Example

- Table Customers

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin	12209	Germany
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.	05023	Mexico
4	Around the Horn	Thomas Hardy	120 Hanover Sq.	London	WA1 1DP	UK
5	Berglunds snabbköp	Christina Berglund	Berguvsvägen 8	Luleå	S-958 22	Sweden

- Delete the customer "Alfreds Futterkiste" from the "Customers" table

Example

- **DELETE FROM** Customers **WHERE** CustomerName='Alfreds Futterkiste’;

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.	05023	Mexico
4	Around the Horn	Thomas Hardy	120 Hanover Sq.	London	WA1 1DP	UK
5	Berglunds snabbköp	Christina Berglund	Berguvsvägen 8	Luleå	S-958 22	Sweden

- **DELETE FROM** Customers;
- deletes all rows in the "Customers" table, without deleting the table.

SELECT Statement

- Used for Retrieving data from a table.

Basic Syntax

- SELECT column1, column2, ... FROM table_name;

- Example:
- SELECT first_name, last_name FROM Employees;

Selecting All Columns

- SELECT * FROM Employees;

- * = all columns
- Quick, but not always efficient
- Best practice → specify only needed columns

Filtering with WHERE

- SELECT first_name, salary FROM Employees WHERE dept_id = 102;
- WHERE is used to specify a condition to filters rows
- Operators: =, >, <, >=, <=, !=
- Logical: AND, OR, NOT
- SELECT emp_id, first_name, last_name, salary, dept_id FROM Employees WHERE dept_id = 102 AND salary > 5000;

Where Clause

- To retrieve all attribute values of any employee who works in DEPARTMENT number 5

```
SELECT *
FROM EMPLOYEE
WHERE Dno = 5;
```

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-09-01	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5

Where Clause

- Retrieve the birth date and address of the employees whose name is 'John B. Smith'

```
SELECT Bdate, Address
FROM employee
WHERE Fname = 'John' and
Minit = 'B' and Lname = 'Smith';
```

(a)

Bdate	Address
1965-01-09	731Fondren, Houston, TX

Where Clause

- To selects all fields from "Customers" where country is "Germany" OR "Spain“

```
SELECT *
FROM Customers
WHERE Country='Germany' OR Country='Spain';
```

- Selects all fields from "Customers" where country is NOT "Germany“

```
SELECT *
FROM Customers
WHERE NOT Country='Germany';
```

IN Operator

- IN Syntax

```
SELECT column_name(s)
 FROM table_name
 WHERE column_name IN (value1, value2, ...);
```

OR

```
 SELECT column_name(s)
 FROM table_name
 WHERE column_name IN (SELECT STATEMENT);
```

- To selects all customers that are located in "Germany", "France" or "UK“

```
 SELECT * FROM Customers
 WHERE Country IN ('Germany', 'France', 'UK');
```

IN Operator Examples

- To selects all customers that are NOT located in "Germany", "France" or "UK“

```
SELECT *
FROM Customers
WHERE Country NOT IN ('Germany', 'France', 'UK');
```

- To selects all customers that are from the same countries as the suppliers

```
SELECT *
FROM Customers
WHERE Country IN (SELECT Country FROM Suppliers);
```

Sorting with ORDER BY

- SELECT first_name, salary FROM Employees ORDER BY salary DESC;
- ASC = ascending (default)
- DESC = descending

Distinct to Remove the Duplicates

- **Retrieve the salary of every employee.**

```
SELECT      Salary

FROM EMPLOYEE;
```

- **Retrieve all distinct salary values.**

```
SELECT DISTINCT Salary

FROM EMPLOYEE;
```

(a)

Salary
30000
40000
25000
43000
38000
25000
25000
55000

(b)

Salary
30000
40000
25000
43000
38000
55000

IS NULL operator

- **Syntax to test for Null value:**

```
SELECT column_names
FROM table_name
WHERE column_name IS NULL;
```

- **Retrieve the names of all employees who do not have supervisors**

```
SELECT Fname, Lname
FROM EMPLOYEE
WHERE Super_ssn IS NULL;
```

(d)

Fname	Lname
James	Borg

Substring matching

- LIKE Operator + % Wildcard
- % means **any sequence of characters (including empty)**.
- _ (underscore) means **exactly one character**.
- SELECT column1, column2 FROM table_name WHERE column_name LIKE 'pattern';

Examples

- SELECT first_name, last_name FROM Employees WHERE first_name LIKE 'A%';
- Match names starting with A
- Matches: Alice, Alan
- Doesn’t match: Bob, Charlie

Example

- SELECT first_name, last_name FROM Employees WHERE last_name LIKE '%son’;

- Match names ending with "son"
- Matches: Johnson, Anderson

Example

- SELECT first_name, last_name FROM Employees WHERE first_name LIKE '%ar%’;
- Match names containing ar
- Matches: Charlie, Parker

Examples

- SELECT first_name FROM Employees WHERE first_name LIKE ‘S____’;
- Match exactly 5-letter names starting with "S"
- Matches: Smith
  Doesn’t match: Sam, Sandra

Between Operator

- Used to filter rows **within a range** (inclusive).
- Works with **numbers, dates, and text**.
- Equivalent to writing >= and <=.

- **Syntax**
- SELECT column1, column2 FROM table_name WHERE column_name BETWEEN value1 AND value2;

Examples

- SELECT first_name, salary FROM Employees WHERE salary BETWEEN 5000 AND 7000;
- Matches salaries 5000, 6000, 7000
- Excludes 4500, 7500
- SELECT first_name FROM Employees WHERE first_name BETWEEN 'A' AND 'M';
- Matches names alphabetically from A → M
- Excludes names starting with N → Z

Functions used with Select

- **1. SUM()**
- Adds up all the values in a numeric column.
- **2. MAX()**
- Finds the largest value in a column.
- **3. MIN()**
- Finds the smallest value in a column.
- **4. AVG()**
- Finds the average value in a numeric column

Example

```
SELECT
    SUM(salary) AS total_salary,
    MAX(salary) AS highest_salary,
    MIN(salary) AS lowest_salary
    AVG(salary) AS avg_salary
FROM Employees;
```

SUM with where clause

- Find Total salary for department **102 only**:
- SELECT SUM(salary) FROM Employees WHERE dept_id = 102;
- Filters rows by dept_id = 102 first
- then adds salaries.
- Note: Functions can’t be used directly in where clause.

Example

- SELECT SUM(salary) FROM Employees WHERE SUM(salary) > 10000;
- Will not work.

COUNT function

- COUNT Syntax

```
SELECT COUNT(column_name)
FROM table_name
WHERE condition;
```

- The COUNT() function returns the number of rows that matches a specified criteria.

Examples

- **Count All Rows**
- SELECT COUNT(*) FROM Employees;
- Returns the total number of rows in the table. (even if some columns have NULL values).
- **Count with a Condition (WHERE)**
- SELECT COUNT(*) FROM Employees WHERE dept_id = 102;
- Returns the number of employees in department 102.

- SELECT COUNT(dept_id) FROM Employees;
- Counts only those rows where there are no Null values for dept_id

- COUNT(*) = counts all rows
- COUNT(column) = counts only rows with **non-NULL values**.

GROUP by

- The GROUP BY statement groups rows that have the same values into summary rows, like "find the number of customers in each country".
- Syntax

```
SELECT column_name(s)
FROM table_name
WHERE condition
GROUP BY column_name(s)
ORDER BY column_name(s);
```

- The GROUP BY statement is often used with aggregate functions (COUNT, MAX, MIN, SUM, AVG) to group the result-set by one or more columns.

GROUP by Examples

- To lists the number of customers in each country

```
SELECT COUNT(CustomerID), Country
FROM Customers
GROUP BY Country;
```

- To lists the number of customers in each country, sorted high to low

```
SELECT COUNT(CustomerID), Country
FROM Customers
GROUP BY Country
ORDER BY COUNT(CustomerID) DESC;
```

Example

```
SELECT Dno, COUNT(*),
AVG(Salary)
FROM EMPLOYEE
GROUP BY Dno;
```

(a)

Fname	Minit	Lname	Ssn	...	Salary	Super_ssn	Dno
John	B	Smith	123456789		30000	333445555	5
Franklin	T	Wong	333445555		40000	888665555	5
Ramesh	K	Narayan	666884444		38000	333445555	5
Joyce	A	English	453453453	...	25000	333445555	5
Alicia	J	Zelaya	999887777		25000	987654321	4
Jennifer	S	Wallace	987654321		43000	888665555	4
Ahmad	V	Jabbar	987987987		25000	987654321	4
James	E	Bong	888665555		55000	NULL	1

Grouping EMPLOYEE tuples by the value of Dno

Dno	Count (*)	Avg (Salary)
5	4	33250
4	3	31000
1	1	55000

Result of Q24

HAVING

- Syntax

```
SELECT column_name(s)
FROM table_name
WHERE condition
GROUP BY column_name(s)
HAVING condition
ORDER BY column_name(s);
```

- The HAVING clause was added to SQL because the WHERE keyword could not be used with aggregate functions(COUNT, MAX, MIN, SUM, AVG).

HAVING Examples

- To lists the number of customers in each country. Only include countries with more than 5 customers:

```
SELECT COUNT(CustomerID), Country
FROM Customers
GROUP BY Country
HAVING COUNT(CustomerID) > 5;
```

- lists the number of customers in each country, sorted high to low (Only include countries with more than 5 customers)

```
SELECT COUNT(CustomerID), Country
FROM Customers
GROUP BY Country
HAVING COUNT(CustomerID) > 5
ORDER BY COUNT(CustomerID) DESC;
```

Retrieving data from more than one table

- Using Join statement – next lecture